MULTITHREADED CHAT APPLICATION USING JAVA

1. Abstract

The Multithreaded Chat Application is a network-based communication system developed using Java Socket Programming and Multithreading concepts. The application consists of a server and multiple clients that can communicate with each other in real time. The server listens for incoming client connections and creates a separate thread for each connected client. This allows multiple users to communicate simultaneously without interrupting each other's communication.

The project demonstrates the practical implementation of networking, sockets, threads, and concurrent programming in Java.

2. Objective

The objectives of this project are:

- To understand Socket Programming in Java.
- To learn Multithreading concepts.
- To implement real-time communication between clients.
- To develop a client-server architecture.
- To manage multiple client connections simultaneously.

3. Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB or above
- Storage: 500 MB free space

Software Requirements

- Operating System: Windows/Linux/MacOS
- Java Development Kit (JDK 17)
- Command Prompt (CMD)
- Notepad or any Java IDE

4. Introduction

Communication systems are widely used in modern applications such as WhatsApp, Telegram, Discord, and Slack. These applications allow multiple users to communicate simultaneously through a network.

Java provides networking support through the java.net package and multithreading support through the java.lang package. By combining these features, a chat application can be developed where multiple clients connect to a server and exchange messages in real time.

5. Concepts Used

Socket

A Socket is an endpoint for communication between two machines.

ServerSocket

A ServerSocket waits for client connection requests.

Thread

A Thread is a lightweight process that enables concurrent execution.

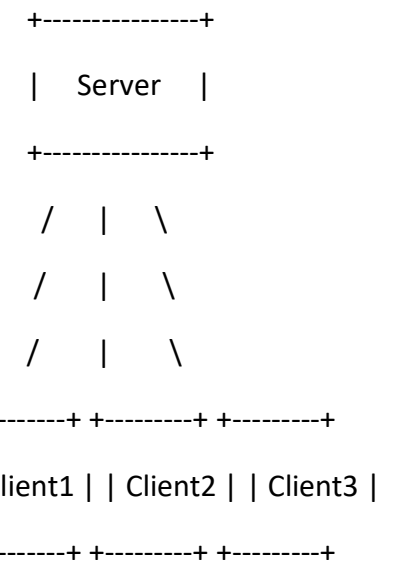
Multithreading

Multithreading allows multiple tasks to execute simultaneously.

Input and Output Streams

Used for sending and receiving messages between server and clients.

6. System Architecture



The server accepts connections from multiple clients and creates separate threads for handling each client.

7. Algorithm

Server Side

1. Create a ServerSocket on a specific port.
2. Wait for client connections.
3. Accept incoming client requests.
4. Create a new thread for each connected client.
5. Receive messages from clients.
6. Broadcast messages to all connected clients.

Client Side

1. Connect to the server using Socket.
2. Create input and output streams.
3. Send messages to the server.
4. Receive messages from the server.
5. Display received messages.

8. Source Code

Chat Server.java

```
import java.io.*;
import java.net.*;


public class ChatServer {


   public static void main(String[] args) {


      try {
```

```
ServerSocket serverSocket = new ServerSocket(5000);

System.out.println("Server started...");
System.out.println("Waiting for client...");

Socket socket = serverSocket.accept();

System.out.println("Client connected!");

BufferedReader input =
      new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

PrintWriter output =
      new PrintWriter(socket.getOutputStream(), true);

BufferedReader console =
      new BufferedReader(
            new InputStreamReader(System.in));

Thread receiveThread = new Thread(() -> {

   try {

      String message;

      while ((message = input.readLine()) != null) {
```

```
                System.out.println("Client: " + message);
            }

        } catch (Exception e) {
            System.out.println("Connection closed.");
        }
    });

    receiveThread.start();

    String serverMessage;

    while ((serverMessage = console.readLine()) != null) {
        output.println(serverMessage);
    }

    socket.close();
    serverSocket.close();

  } catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
  }
 }
}
```

Chatclient.java

```
import java.io.*;
import java.net.*;
```

```
public class ChatClient {

    public static void main(String[] args) {

        try {

            Socket socket = new Socket("localhost", 5000);

            System.out.println("Connected to Server!");

            BufferedReader input =
                    new BufferedReader(
                            new InputStreamReader(socket.getInputStream()));

            PrintWriter output =
                    new PrintWriter(socket.getOutputStream(), true);

            BufferedReader console =
                    new BufferedReader(
                            new InputStreamReader(System.in));

            Thread receiveThread = new Thread(() -> {

                try {

                    String message;

                    while ((message = input.readLine()) != null) {
```

```
                System.out.println("Server: " + message);
            }

        } catch (Exception e) {
            System.out.println("Connection closed.");
        }
    });

    receiveThread.start();

    String clientMessage;

    while ((clientMessage = console.readLine()) != null) {
        output.println(clientMessage);
    }

    socket.close();

  } catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
  }
 }
}
```

9. Sample Output

Server Window

Server started...

Client connected.

Client connected.

Message received from Client 1:
Hello Everyone

Message received from Client 2:
Hi Client 1

Client 1 Window

Connected to server.

Hello Everyone

Hi Client 1

Client 2 Window

Connected to server.

Hello Everyone

Hi Client 1

```
C:\Users\panak\OneDrive\Desktop\Task3_Multithreaded_Chat>javac ChatServer.java

C:\Users\panak\OneDrive\Desktop\Task3_Multithreaded_Chat>java ChatServer
Server started...
Waiting for client...
Client connected!
hello client
Client: hello server
```

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\panak>cd Onedrive\Desktop\Task3_Multithreaded_Chat

C:\Users\panak\OneDrive\Desktop\Task3_Multithreaded_Chat>java ChatClient
Connected to Server!
Server: hello client
hello server
```

10. Applications

- Online Chat Systems
- Customer Support Systems
- Team Collaboration Tools
- Messaging Applications

- Multiplayer Gaming Platforms
- Social Networking Applications

11. Advantages

- Supports multiple clients simultaneously.
- Real-time communication.
- Efficient use of system resources.
- Scalable architecture.
- Demonstrates networking concepts effectively.

12. Limitations

- Requires network connectivity.
- Security features are not implemented.
- Data is transmitted in plain text.
- Suitable mainly for educational purposes.

13. Future Enhancements

- User Authentication
- Encrypted Communication
- Graphical User Interface (GUI)
- File Sharing
- Voice and Video Communication
- Database Integration

14. Conclusion

The Multithreaded Chat Application successfully demonstrates the implementation of client-server communication using Java Socket Programming and Multithreading. The server handles multiple client connections simultaneously, enabling real-time message exchange.

This project provides a strong foundation for understanding networking concepts and developing large-scale communication applications.

15. Viva Questions and Answers

Q1. What is Socket Programming?

Socket Programming is a method of communication between two devices over a network using sockets.

Q2. What is a Socket?

A Socket is an endpoint used for sending and receiving data over a network.

Q3. What is ServerSocket?

ServerSocket is a Java class used to create a server that listens for client requests.

Q4. What is Multithreading?

Multithreading is the execution of multiple threads simultaneously within a program.

Q5. Why is Multithreading used in a chat application?

To handle multiple client connections concurrently.

Q6. What is a Thread?

A Thread is the smallest unit of execution within a process.

Q7. What package is used for networking in Java?

java.net package.

Q8. What package is used for input/output operations?

java.io package.

Q9. What is Client-Server Architecture?

A model where clients request services and the server provides them.

Q10. What are the advantages of multithreading?

Improved performance, better resource utilization, and simultaneous task execution.